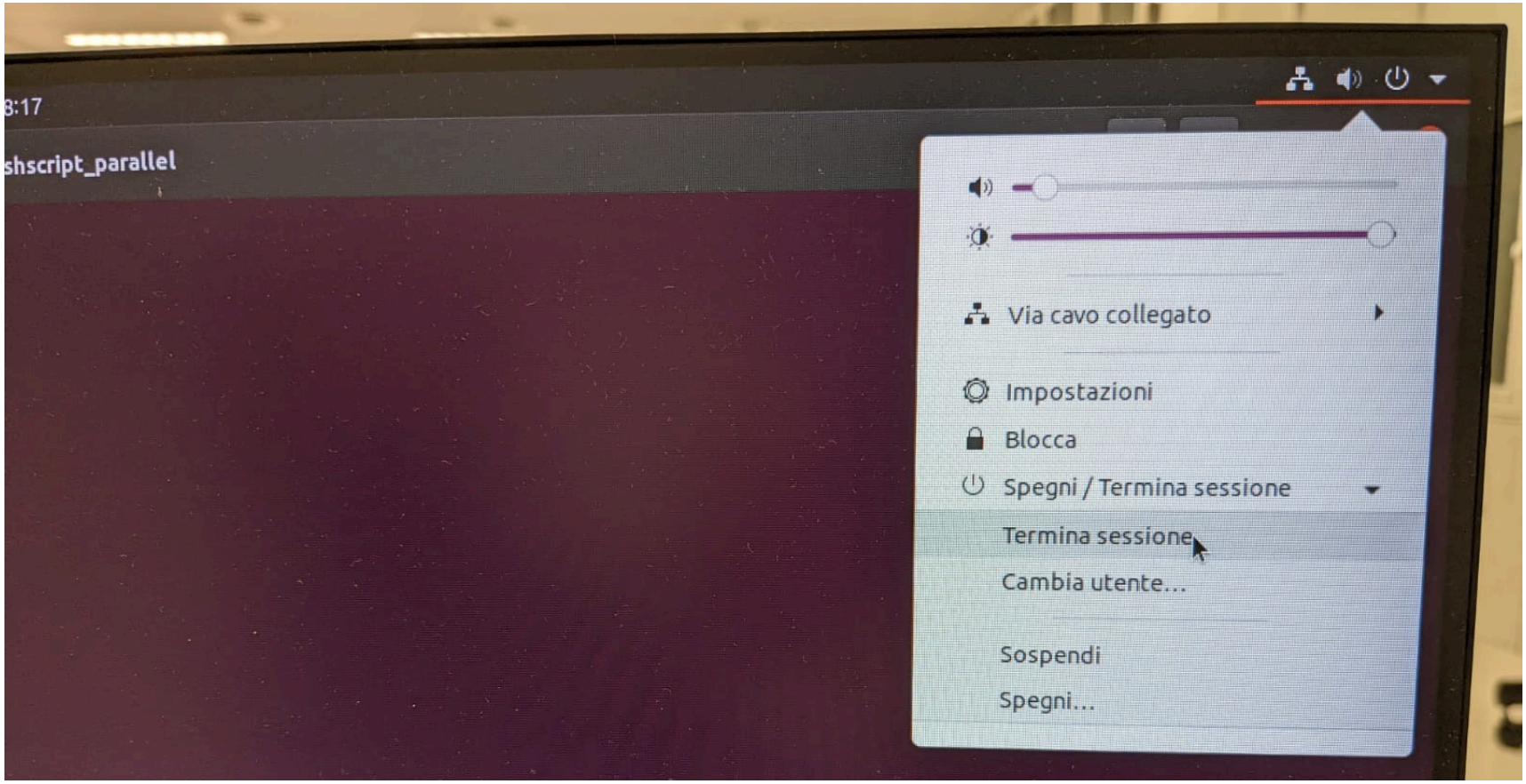
Corso di Sistemi Operativi

Corso estinto di Sistemi Operativi e Reti - Modulo Sistemi Operativi

Prova scritta - Giugno 2026

LEGGI ATTENTAMENTE LE ISTRUZIONI:

1. **Rinomina** subito la cartella chiamata "Cognome-Nome-Matricola" che hai trovato sul Desktop e in cui hai trovato questa traccia, sostituendo "Cognome" "Nome" e "Matricola" con i tuoi dati personali e **lasciando i trattini senza spazi**; se hai un doppio nome oppure un doppio cognome dovrai chiamare la cartella come in questo esempio:
    a. DeLuca-MarcoGiovanni-199999
2. **Carica** tutto il materiale didattico che vorrai usare sul Desktop; puoi farlo solo nei primi 5 minuti della prova;
3. **Svolgi** il compito; lascia tutto il sorgente che hai prodotto nella cartella di cui al punto 1;
4. Quando hai finito fai "**Termina Sessione/Logout**", ma lascia la postazione mantenendo il PC acceso.
5. **E' tua diretta responsabilità** garantire l'integrità del tuo elaborato, anche in caso di assenza di corrente. **Salva spesso il tuo lavoro**



e NON spegnere il PC.

SALVA SPESSO

CI SONO DEI PUNTI AMBIGUI NELLA TRACCIA? DECIDI TU COSA FARE

È parte integrante di questo esercizio completare le specifiche date nei punti non esplicitamente definiti, introducendo nuove strutture dati, o estendendo quelle preesistenti laddove si ritenga necessario, risolvendo eventuali ambiguità. Si può cambiare il codice dei metodi esistenti dove serve.

POSSO CAMBIARE IL PROTOTIPO DEI METODI RICHIESTI O DI QUELLI ESISTENTI? Si, ma solo se li lasci retrocompatibili

Il codice che scriverai è pensato per essere usato come un modulo di libreria. Un modulo di libreria potrebbe essere usato da altri programmatori, i quali si aspettano di trovare una specifica interfaccia.

Se quindi modifichi il prototipo dei metodi pubblici di una classe, devi assicurarti che possano essere comunque utilizzati per come erano stati pensati prima delle tue modifiche. Puoi aggiungere qualsivoglia campo e metodo privato, e qualsivoglia classe ausiliaria. Analogamente, i metodi esistenti possono essere modificati nel loro codice, ma non se ne deve cambiare il risultato finale o il significato a meno che non ti venga richiesto esplicitamente.

CHE LINGUAGGIO POSSO USARE? PYTHON 3.X

POSSO CONSENTIRE SITUAZIONI DI RACE CONDITION NEL MIO CODICE? NO
POSSO CONSENTIRE SITUAZIONI DI DEADLOCK NEL MIO CODICE? NO
POSSO CONSENTIRE ALTRE SITUAZIONI DI BLOCCO TOTALE NEL MIO CODICE, TIPO NESTED LOCKOUT, LIVELOCK O ALTRO? NO
POSSO CONSENTIRE SITUAZIONI DI STARVATION NEL MIO CODICE? SI, tranne quando ti viene chiesto esplicitamente di rimuoverle
MA IL MAIN() LO DEVO AGGIORNARE? E I THREAD DI PROVA? SI

E' obbligatorio implementare esplicitamente il codice di prova oppure modificare il codice di prova pre-esistente, e accertarsi che giri senza errori prima della consegna.

ESERCIZIO 1 - PROGRAMMAZIONE MULTITHREADED

(Punteggio minimo richiesto 18/30. Pesa per ⅔ del voto finale)

Svolgi tutti i punti della traccia in un unico file che chiamerai `SOLUZIONE.py`
`-> Qualsiasi altro nome di file verrà IGNORATO <-`

Punto 1

Introduci il concetto di chilometraggio percorso da ciascun furgoncino per ogni tratta. Un furgoncino viene considerato vecchio quando i suoi chilometri percorsi eccedono il valore di un valore globale `km_max`, e deve essere dunque ritirato dal ciclo produttivo nel momento opportuno e senza perdere pacchi.

Punto 2

Introduci dei metodi che consentano di *chiudere* o *aprire* una specifica sede. Quando una sede è aperta, questa si comporta normalmente. Se una sede è *chiusa*, i furgoncini che dovessero arrivare in quel momento devono proseguire verso la sede successiva senza essere scaricati. Per implementare questa cosa, puoi considerare l'idea di fare passare direttamente il furgoncino da un guidatore al guidatore successivo, cosa che potresti ottenere facilmente trovando il modo di spostare i furgoncini direttamente dal garage di arrivo a quello di partenza saltando la fase di scarico. Decidi tu come al solito come risolvere eventuali altre difficoltà di realizzazione che dovessero presentarsi.

Punto 3

Prevedi un thread che tenga conto di una variabile `furgoncini_min`, e che immette nel sistema delle sedi 5 nuovi furgoncini ogni qual volta il numero totale di furgoncini che circolano in tutto il sistema scende sotto `furgoncini_min`.

Punto 4

Implementa un thread `Stopper` e creane una istanza. Allo scadere di un valore detto `tempo_di_vita` il Thread in questione deve fare in modo che avvengano le seguenti cose: *1)* Tutti i thread produttori di pacchi cessano la produzione; *2)* I Guidatori continuano nel loro ciclo di vita, ma terminano quando si troveranno a guidare un furgoncino vuoto; *3)* I Lavoratori continuano nel loro ciclo di vita, ma terminano, facendo partire il furgoncino corrente, allorquando non trovano prodotti nel buffer temporaneo; *4)* Infine, opzionalmente visto che ti ho detto che non dovevi studiare come funziona il display, il display cessa di stampare e il programma termina.

SALVA SPESSO

ESERCIZIO 2 - LINGUAGGI DI SCRIPTING

(Punteggio minimo richiesto 18/30. Pesa per ⅓ del voto finale)

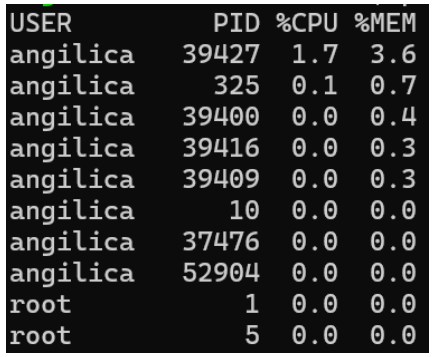
Trovi allegato a questa traccia uno script Perl che presenta errori di sintassi e/o logici. Sistemalo per ottenere il comportamento che ti descriviamo sotto. ***NON devi stravolgere il contenuto dello script o riscriverlo da capo: la sua struttura deve rimanere pressoché invariata. E' prevista una penalità se correggi parti di codice che invece non presenta bug.***

Lo script `info_giu26.pl` dovrebbe essere in grado di fornire dettagli riguardanti l'utilizzo delle risorse (CPU o memoria) da parte di uno specifico utente.
In particolare, lo script riceve come argomenti da linea di comando esattamente 2 opzioni: la prima identifica il tipo di risorsa da andare a consultare (parametro `-c` per consultare la CPU, parametro `-m` per consultare la memoria) mentre la seconda identifica l'utente da filtrare. Lo script dovrà essere eseguito, quindi, con la seguente sintassi:

```
perl info_giu26.pl [-c|-m] username
```

Ad ogni esecuzione, lo script deve appendere a un file chiamato `stat.log` una riga contenente la data e l'orario corrente. il nome dell'utente inserito come parametro dello script e la percentuale totale di CPU (o di memoria) utilizzata. Per conoscere la data e l'orario corrente si può usare il comando `date`.

Esempio:
Immagina di avere uno scenario come quello mostrato di seguito



```
USER         PID %CPU %MEM
angilica   39427  1.7  3.6
angilica     325  0.1  0.7
angilica   39400  0.0  0.4
angilica   39416  0.0  0.3
angilica   39409  0.0  0.3
angilica      10  0.0  0.0
angilica   37476  0.0  0.0
angilica   52904  0.0  0.0
root           1  0.0  0.0
root           5  0.0  0.0
```

se lo script fosse invocato il 24 Giugno 2026 alle 11:00 come

```
perl info_giu26.pl -m angilica
```

allora dovresti vedere che è stato appeso al file `stats.log` la seguente stringa.

```
Wed Jun 24 11:00:27 CEST 2026 %MEM angilica 5.3
```

se invece lo stesso script, alla stessa ora, fosse invocato come

```
perl info_giu26.pl -c angilica
```

allora dovrebbe essere aggiunta al file di log la riga:

```
Wed Jun 24 11:00:27 CEST 2026 %CPU angilica 5.3
```

N.B. Non è un problema se data e orario vengono stampati in italiano invece che in inglese.